1 2 8

fixtures/alias_spread/ — T self-validation fixtures

Fixture convention for `constitution/scripts/fastcycle/orchestration/limit_class.py place` (plan.md T-B07; tasks.md T128/T136; research.md DEC-22; FR-016, FR-018). Task T128 (`tasks.md`) writes ONLY this RED test + these fixtures; the tool itself is T136, a later, separate task (Producer != Verifier, section 11.4.240 -- this file's author never implements `place`).

Where this sits relative to the contract that DOES exist

Unlike T109/T118's `context/tier_route.py` (which has NO contract file at all -- `common-conventions.md` 's own "Tool map" gap list names it explicitly), `limit_class.py` IS covered by a real contract: `contracts/agent-registry-and-handoff.md` . That contract's Components block already fixes ONE of `limit_class.py` 's two verbs:

```
$FC/orchestration/limit_class.py --signal <raw> --out <class.json>
```

That is `classify` (T-B06/T127/T135) -- a DIFFERENT verb from this file's concern. `agent-registry-and-handoff.md` defines no `place` verb, no `place` CLI shape, and no `place` -specific RED fixture anywhere in its own RED fixtures table -- every one of its listed fixtures is prefixed `ar_` or `ho_` and concerns registry/handoff lifecycle, none concerns alias spreading. So `place` (T-B07's content) is an UNDER-SPECIFIED CORNER of an EXISTING contract -- the T104/

T108 situation (contract exists, corner undefined), not the T109/T118 situation (no contract exists at all). This file DEFINES that corner -- CLI verb shape included -- from plan.md T-B07 + research.md DEC-22

- tasks.md T128/T136's own wording alone, flagged explicitly as binding-if-adopted, never claimed as something the contract already fixed.

The chosen CLI shape reuses the `--fixture <path> --out <path>` convention already established for `context/tier_route.py route` (T109/T118, the closest sibling tool in this codebase: another orchestration-layer decision function consuming a JSON fixture and emitting a JSON verdict document) rather than inventing a third shape:

```
limit_class.py place --fixture <fixture.json> --out <placement.json>
```

Adjustable at T136 review time if T136 chooses a different verb/shape, mirroring `common-conventions.md` 's own "no contract exists; fixed by task text until a contract is written" convention applied here one level further down (an under-specified verb of an existing contract, not a whole missing contract).

The placement rule this fixture set encodes (from plan.md T-B + research.md

DEC- , restated precisely, never invented)

DEC-22's decision text, verbatim (`specs/004-fast-dev-cycles/research.md` , section "DEC-22 -- Spread fan-out across aliases"):

> the dispatcher assigns concurrent agents to aliases so that no alias carries more than `ceil(live_agents / operational_aliases)` + 1 agents, honouring section 11.4.196 native-first order; an alias within a configurable margin of a known cap (weekly reset recorded by DEC-14) receives no new long-running agents.

Three clauses this oracle implements, in order:

1. **Eligibility filter.** An alias is ELIGIBLE for new placements iff it is `operational: true` AND `near_cap: false` (DEC-22's second clause: "an alias within a configurable margin of a known cap ... receives no new long-running agents" -- `near_cap` is this fixture set's boolean encoding of "within that margin"; the margin's own numeric value is DEC-22's "a configurable margin", UNCONFIRMED and out of this fixture set's scope -- each fixture states the boolean directly as a given classification input, exactly as `tier_route`'s `min_fitting_tier` and `governance_subset`'s `classification_inputs` are given rather than computed by their own RED fixture sets).
2. **Native-first order (section 11.4.196).** The eligible aliases are ordered NATIVE aliases first (in their given relative order), then PROVIDER aliases (in their given relative order) -- a stable partition-by-kind, never a re-sort within a kind. This is genuinely exercised by every fixture below: each fixture's raw `aliases` array is deliberately NOT pre-sorted native-first, so a placement algorithm that merely iterates the raw array order would fail the recorded `native_first_order` field.
3. **Round-robin spread up to a per-alias cap.** With `m` = the eligible alias count and `n` = `live_agents`, the cap per alias is `ceil(n / m) + 1` -- DEC-22's own formula, copied verbatim (never independently re-derived) and confirmed present, right now, in the LIVE `research.md` DEC-22 text by this suite's control needle #4. Agents are assigned one at a time, cycling through the eligible aliases in native-first order (`eligible[i % m]` for the i-th agent, 0-indexed), until all `n` agents are placed. No eligible alias ⇒ refused (an UNCONFIRMED corner -- no fixture in this set exercises `n > 0` with zero eligible aliases; explicitly out of scope, see below).

The named paired mutation this file self-tests (tasks.md T 's own wording: "fill-first placement")

`research.md` DEC-22's own "Alternatives considered" section names this mutation's exact meaning, verbatim (never invented):

> (a) fill the healthiest alias first -- rejected: concentrates blast radius

"Fill-first" = place ALL `live_agents` onto the single FIRST eligible alias (in native-first order), ignoring both the round-robin spread AND the cap entirely. Applied to any fixture where `live_agents` exceeds `cap_per_alias`, this produces `max_assigned_count > cap_per_alias` -- a directly detectable cap violation, proving a real meta-test built against these fixtures, once T136 lands, would genuinely catch that regression.

Fixtures

Fixture	Kind	live_a-gents	eligi-ases
`as_golden_spread.json`	golden (task's own named scenario)	8	4 (2 2 pro mixe orde
`as_negctrl_single_alias.json`	negative control (task's own named scenario)	8	1 (or as op tiona
`as_golden_near_cap_excluded.json`	golden (bonus, beyond this task's min-imum-named scenario per section 11.4.194 all-angle review -- mirrors T109's own `tr_golden_double_escalation.json` precedent for exercising a second expli-cit clause of the same decision)	6	2 of 3 nativ exclu for `nea` `true` pite `ope` `tion` `true`

Explicit scope exclusion (section . . -- honestly out of scope, never

silently absorbed)

Zero-eligible-alias refusal (`n > 0` , every alias either non-operational or near_cap) is UNCONFIRMED by both `tasks.md` T128's own wording and plan.md T-B07's "Protecting tests" clause -- neither names a golden-bad fixture for this tool (unlike T-E06/T109, whose "Protecting tests" clause explicitly named a golden-bad). Left for T136's own implementation decision at review time, exactly as T118's README left ladder-exhaustion out of scope for the analogous reason (no fixture in either task's own wording exercises it). This file's independent oracle (`derive_placement` , below) raises rather than fabricating a verdict for this shape, mirroring `tier_route` 's `derive_route` precedent.